

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

По лабораторной работе №4
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
на тему: «Бинарное дерево поиска»

Выполнили ст.гр.25BBB2:

Кашаев Е.А.

Кривошей В.М.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучить бинарное дерево поиска и реализовать основные операции: построение, вывод, поиск элемента, подсчёт вхождений, исключение дубликатов и оценку сложности поиска.

Ход работы

Задание 1: Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.

Результат работы программы представлен на рисунке 1.

```
-1 - finish building the tree
Enter a number: 2
Enter a number: 14
Enter a number: 10
Enter a number: 18
Enter a number: -1
Tree building finished

2
    10
  14
    18

Enter a value to search for: 2
Value 2 found in the tree (node address: 000002A1C0013C10)
```

Рис.1–результат работы программы к заданию1.

Поиск значения в дереве реализован функцией FindNode, которая ищет узел с заданным значением, начиная с корня, и опирается на свойство бинарного дерева поиска: если искомое значение меньше текущего узла — идём влево, если больше — вправо, при совпадении возвращаем указатель на узел, а при достижении NULL считаем, что значение не найдено. Сложность поиска составляет $O(h)$, где h — высота дерева.

Задание 2: Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.

Результат работы программы представлен на рисунке 2.

```
-1 - finish building the tree
Enter a number: 2
Enter a number: 2
Enter a number: 2
Enter a number: 14
Enter a number: 15
Enter a number: 19
Enter a number: 21
Enter a number: -1
Tree building finished

      2
     / \
    2   14
   / \  / \
  2  15 19 21

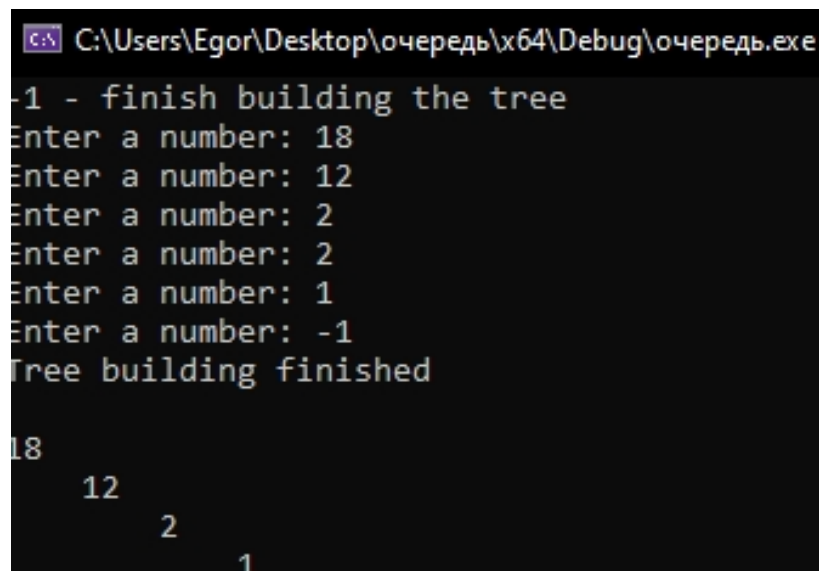
Enter a value to search for: 2
Value 2 found in the tree (node address: 000002459D613FD0)
Value 2 occurs in the tree 3 time(s)
```

Рис.2–результат работы программы к заданию 2.

Подсчёт вхождений элемента выполняется функцией CountOccurrences, которая считает, сколько раз заданное значение встречается в дереве, рекурсивно обходя всё дерево целиком — и левое, и правое поддерево — и прибавляя единицу каждый раз, когда data совпадает с искомым значением. В отличие от поиска, функция не останавливается на первом найденном узле, поэтому её сложность равна $O(n)$, так как в худшем случае приходится обойти все узлы.

Задание 3: Изменить функцию добавления элементов для исключения добавления одинаковых символов.

Результат работы программы представлен на рисунке 3



```
C:\Users\Egor\Desktop\очередь\x64\Debug\очередь.exe
-1 - finish building the tree
Enter a number: 18
Enter a number: 12
Enter a number: 2
Enter a number: 2
Enter a number: 1
Enter a number: -1
Tree building finished

18
 12
   2
    1
```

Рис.3—результат работы программы к заданию 3.

Исключение дубликатов реализовано изменением функции CreateTree, которая теперь не позволяет добавлять в дерево повторяющиеся значения: перед рекурсивным спуском добавлена проверка `if (data == r->data) return root;`, и если такое значение уже есть, новый узел не создаётся. Заодно исправлено условие вставки на классическое для BST (`data < root->data` — влево, иначе вправо). В результате все значения в дереве становятся уникальными, а функция CountOccurrences теперь всегда возвращает либо 0, либо 1.

4. Оценить сложность процедуры поиска по значению в бинарном дереве.

Случай	Высота h	Сложность
Лучший (значение в корне)	—	$O(1)$
Средний (сбалансированное дерево)	$\log_2 n$	$O(\log n)$
Худший (вырожденное дерево, ввод по порядку)	n	$O(n)$

В среднем поиск работает за $O(\log n)$, но без балансировки в худшем случае (например, при вводе чисел по возрастанию) деградирует до $O(n)$.

Вывод: в ходе работы на языке C реализовано бинарное дерево поиска и все required операции: вставка, вывод, поиск, подсчёт вхождений и запрет дубликатов.